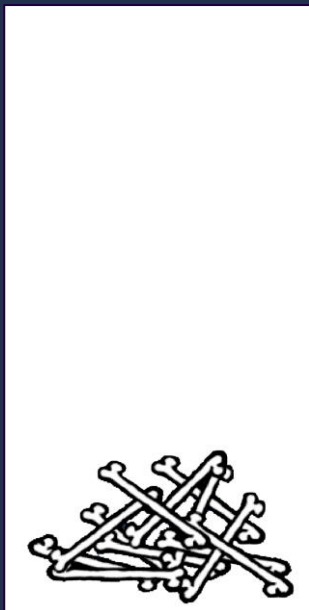
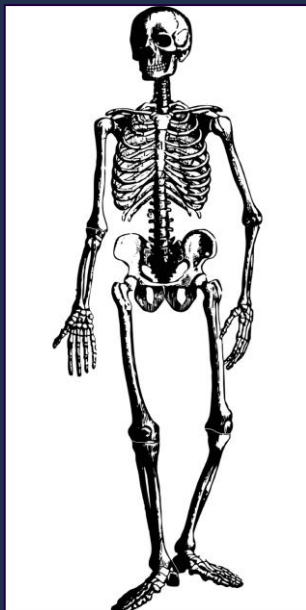
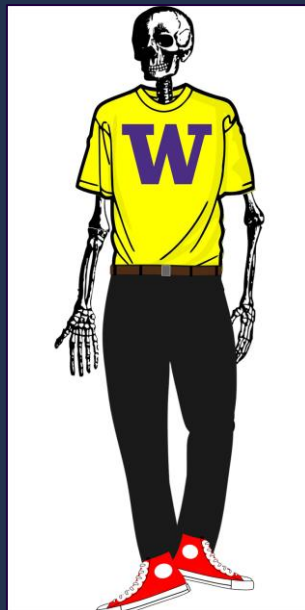




WORDS + IMAGES



HTML



CSS



JAVASCRIPT



FRONTEND FRAMEWORK

Bài 3. CSS

CSS

- Khái niệm:
 - CSS: viết tắt của Cascading Style Sheets
 - Là ngôn ngữ được dùng để định dạng kiểu hiển thị cho các phần tử HTML.
- Cú pháp

```
selector{  
    property: value;  
}
```

- selector: Được gọi là bộ chọn.
- property: Được gọi là thuộc tính.
- value: Được gọi là giá trị của thuộc tính.

CSS

- Bộ chọn:
 - Bộ chọn CSS chọn (các) phần tử HTML mà bạn muốn tạo kiểu.
 - Ví dụ: Các thẻ <h1>, <p>, <table>,... thì viết trong css sẽ chỉ lấy tên thẻ là h1, p, table
- Phân loại:
 - Simple selectors (Bộ chọn đơn giản)
 - Combinator selectors (Bộ chọn tổ hợp)
 - Pseudo-class selectors (Bộ chọn lớp giả)
 - Pseudo-elements selectors (Bộ chọn phần tử giả)
 - Attribute selectors (Bộ chọn thuộc tính)



CSS

- Bộ chọn đơn giản:

Bộ chọn **element** (element selector): chọn các phần tử HTML dựa trên tên phần tử.

```
p {  
    text-align: center;  
    color: red;  
}
```

Bộ chọn **id** (id selector): Sử dụng thuộc tính id (identifier - định danh) của một phần tử HTML để chọn một phần tử cụ thể. Lưu ý: Tên id không được bắt đầu bằng số.

```
#title {  
    text-align: center;  
    color: red;  
}
```

Bộ chọn **class** (class selector): chọn các thành phần HTML với một thuộc tính class cụ thể. Lưu ý: Tên class không được bắt đầu bằng số.

```
.title {  
    text-align: center;  
    color: red;  
}
```



CSS

- Bộ chọn đơn giản:

Bộ chọn chung (universal selector): chọn tất cả các thành phần HTML trên trang.

```
* {  
    text-align: center;  
    color: red;  
}
```

Bộ chọn nhóm (grouping selector): Bộ chọn nhóm chọn tất cả các element, id, class có cùng thuộc tính và giá trị thuộc tính.

```
h1, h2, p {  
    text-align: center;  
    color: red;  
}
```

CSS

- Các kiểu chèn (nhúng) CSS:
 - Inline CSS:
 - Ta sẽ thêm thuộc tính style vào trong thẻ mà muốn sửa CSS.
 - Có thể được sử dụng để áp dụng một kiểu duy nhất cho một element.
 - Ví dụ: `<p style="color:red;">This is a paragraph.</p>`
 - Internal CSS:
 - Ta sẽ thêm thẻ `<style></style>` và bất cứ đâu trong file html, sau đó viết CSS vào trong thẻ `<style>` đó.
 - Thường thì thẻ `<style>` này sẽ nằm trong thẻ `<head></head>`.
 - Có thể được sử dụng nếu có một trang HTML duy nhất.
 - Ví dụ:

```
<style>
p{
    color:red;
}
</style>
```

CSS

- Các kiểu chèn (nhúng) CSS:
 - External CSS:
 - Ta thêm 1 file css ở bên ngoài, sau đó dùng thẻ `<link />` đặt ở trong thẻ `<head></head>` và chèn đường dẫn vào thuộc tính href ở trong thẻ link.
 - Có thể thay đổi giao diện của toàn bộ trang web bằng cách chỉ thay đổi một tệp.
 - Ví dụ: `<link rel="stylesheet" href="mystyle.css">`
- Lưu ý: Nếu trùng bộ chọn và tên thuộc tính, thì giá trị của bộ chọn cuối cùng được sử dụng.

```
h1 {  
    color: navy;  
}  
h1 {  
    color: orange;  
}
```

→ Màu được chọn sẽ là màu orange.

CSS

- Các thuộc tính CSS căn bản:
 - Màu sắc (**color**):
 - Cài đặt phần mềm Just Color Picker
 - Colors: Màu sắc được gán giá trị bằng cách sử dụng:
 - Một số màu được đặt tên sẵn: white, black, red, green, blue, yellow, orange,... (ít sử dụng)
 - HEX (Phổ biến nhất)
 - RGB (ít sử dụng)
 - HSL (cực ít sử dụng)

HEX (Phổ biến nhất)

Màu HEX được chỉ định bằng: #RRGGBB, trong đó các số nguyên thập lục phân RR (đỏ), GG (xanh lục) và BB (xanh dương) chỉ định các thành phần của màu.

Cú pháp: #RRGGBB

Ví dụ: #000000 (màu đen), #FFFFFF (màu trắng), #FF0000 (màu đỏ)

```
h1{  
    color: #FF0000;  
}
```



CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background-color: nền là màu sắc
 - background-clip: xác định phạm vi được thiết lập màu nền của phần tử (Áp dụng cho background là màu sắc)
 - border-box: Mặc định. Đổ màu từ content cho đến hết border.
 - padding-box: Đổ màu từ content cho đến hết padding.
 - content-box: Chỉ đổ màu phần content.

☐ `background-clip: border-box;`

☐ `background-clip: padding-box;`

☒ `background-clip: content-box;`

Content goes here.

CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background-image: nền là hình ảnh hoặc màu gradient.
 - Giá trị là kiểu hình ảnh:
 - background-image: url("hinh-anh.jpg");
 - Có thể chèn nhiều hình ảnh: (url nào ở trước thì sẽ nằm bên trên)
 - background-image: url("hinh-1.png"), url("hinh-2.jpg");
 - Có thể là kiểu gradient: Tạo ra một dải màu chuyển đổi từ màu này đến màu khác.
 - Ví dụ: background-image: linear-gradient(180deg, #000000, #ffffff);
 - Kết hợp cả hình ảnh và gradient:
 - Ví dụ: background-image: linear-gradient(180deg, #00000080, #ffffffad), url("hinh-anh.jpg");

CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background-size: Kích thước của background.
 - Giá trị có thể là đơn vị (px, %, ...).
 - Ví dụ: background-size: 100% auto;
 - contain: sẽ co giãn hình ảnh để hình ảnh nằm trọn trong khung element, hình ảnh không bị vỡ.
 - Ví dụ: background-size: contain;
 - cover: kéo giãn hình ảnh sao cho vừa với khung, cắt bỏ đi những phần ảnh thừa để hình ảnh không bị vỡ.
 - Ví dụ: background-size: cover;



CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background-repeat: nền được lặp lại hay không.
 - Ví dụ:
 - background-repeat: no-repeat; // Không lặp lại ảnh
 - background-repeat: repeat-x; // Lặp theo trục ngang
 - background-repeat: repeat-y; // Lặp theo trục dọc
 - background-origin: giống background-clip, nhưng background-clip là dành cho nền là màu sắc, còn background-origin là dùng cho nền ảnh.
 - content-box: background chỉ chiếm phần content.
 - padding-box: background chiếm phần content và padding.
 - border-box: background chiếm phần content, padding và border.

CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background-position: vị trí hình nền so với element. Có các giá trị: top, left, right, bottom, center.
 - Ví dụ:
 - background-position: top left; // trên - trái
 - background-position: top right; // trên - phải
 - background-position: top center; // trên - giữa
 - background-position: bottom left; // dưới - trái
 - background-position: bottom right; // dưới - phải
 - background-position: bottom 30px right 20px; // cách dưới 30px - cách phải 20px
 - background-attachment: nền sẽ được cuộn hoặc cố định.
 - Ví dụ:
 - background-attachment: fixed; // Hình nền được fix cố định khi di cuộn web



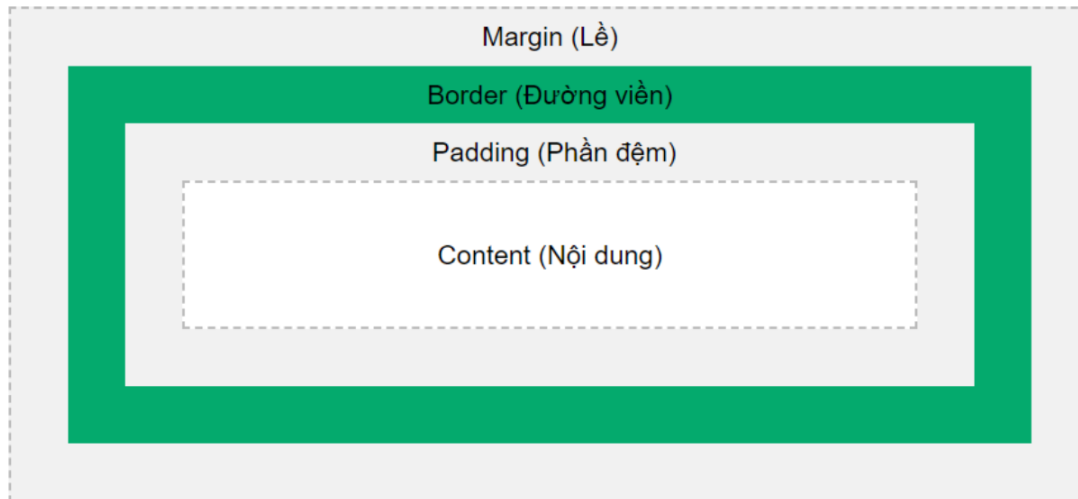
CSS

- Các thuộc tính CSS căn bản:
 - Màu nền (background-color):
 - background (cách viết ngắn).
 - Ví dụ:
 - background: #ffffff url("hinh-anh.png") no-repeat top right;



CSS

- Các thuộc tính CSS căn bản:
 - Box Model (Mô hình hộp):
 - Tất cả các element có thể được coi là các cái hộp.
 - Hộp này bao gồm: lề, đường viền, phần đệm và nội dung.
 - Vào dev tools chọn tab Computed sẽ thấy sơ đồ của một element dùng để biểu diễn cho element ta đã chọn.



Content (Nội dung) - Nội dung của element, nơi văn bản và hình ảnh xuất hiện.

Padding - Tạo một khoảng trống xung quanh nội dung. Phần đệm trong suốt.

Border (Đường viền): Đường viền bao quanh padding (phần đệm) và nội dung.

Margin (Lề) - Tạo khoảng cách cho vùng bên ngoài border (đường viền). Lề trong suốt.

CSS

- Các thuộc tính CSS căn bản:
 - Đường viền (Border):
 - Đường viền bao quanh padding (phần đệm) và nội dung. Dùng để bao bọc 1 element
 - Tên thuộc tính: border-style
 - Các kiểu border:
 - dotted - Đường viền chấm
 - dashed - Đường viền nét đứt
 - solid - Đường viền liền
 - double - Đường viền kép
 - groove - Đường viền có rãnh 3D
 - ridge - Đường viền có gờ 3D
 - inset - Đường viền 3D chìm vào trong.
 - outset - Đường viền 3D nổi lên trên.
 - none - Không có viền
 - hidden - Đường viền bị ẩn

```
p {  
    border: 5px solid red;  
}
```


04. Box Model, Borders, Padding, Margins

4.3. Padding (Phần đệm)

Padding (phần đệm) tạo một khoảng trống xung quanh nội dung, và nằm bên trong đường viền.

Ví dụ:

```
div {  
  padding-top: 50px;  
  padding-right: 30px;  
  padding-bottom: 50px;  
  padding-left: 80px;  
}
```

```
div {  
  padding: 25px;  
}
```

```
div {  
  padding: 25px 50px;  
}
```

04. Box Model, Borders, Padding, Margins

4.4. Margins (Lề)

Margin (Lề) để tạo khoảng cách giữa element này với element khác. Margin nằm bên ngoài

border.

Ví dụ:

```
p {  
  margin-top: 100px;  
  margin-bottom: 100px;  
  margin-right: 150px;  
  margin-left: 80px;  
}
```

```
p {  
  margin: 25px;  
}
```

```
p {  
  margin: 25px 50px;  
}
```

05. Thuộc tính height, width

height: Chiều cao của element

width: Chiều rộng của element

min-height: chiều cao tối thiểu của element

min-width: chiều rộng tối thiểu của element

max-height: chiều cao tối đa của element

max-width: chiều rộng tối đa của element

Lưu ý: Chiều cao và chiều rộng là của phần content nằm bên trong phần padding.

Không bao gồm padding, border, margin



06. Text, Fonts, Icons

6.1. Text

Text Color: thuộc tính color được sử dụng để đặt màu cho văn bản.

↴ Ví dụ: color: green;

- Text Align: được sử dụng để thiết lập căn lề ngang của văn bản.
 - text-align: center; // Căn giữa
 - text-align: left; // Căn trái
 - text-align: right; // Căn phải
 - text-align: justify; // Căn đều 2 bên

06. Text, Fonts, Icons

6.1. Text

Text Transform: thuộc tính text-transform được sử dụng để chỉ định chữ hoa và chữ thường trong văn bản.

- ↴ text-transform: uppercase; // VIẾT CHỮ HOA
- ↴ text-transform: lowercase; // viết chữ thường
- ↴ text-transform: capitalize; // Viết Hoa Các Chữ Cái Đầu

06. Text, Fonts, Icons

6.2. Fonts

Trong CSS có năm họ phong chữ:

- ↴ **Phông chữ Serif** (có chân) có một nét nhỏ ở các cạnh của mỗi chữ cái.
- ↴ **Phông chữ Sans-serif** (không chân) có đường kẻ rõ ràng (không có nét nhỏ đi kèm).
- ↴ Phông chữ Monospace (Đơn cách) - ở đây tất cả các chữ cái có cùng chiều rộng cố định.
- ↴ Phông chữ Cursive (Chữ viết ầu) bắt chước chữ viết tay của con người.
- ↴ Phông chữ Fantasy (tưởng tượng) là phông chữ trang trí/vui tươi.

06. Text, Fonts, Icons

6.2. Fonts

Sự khác biệt giữa phong chữ Serif và Sans-serif:



06. Text, Fonts, Icons

6.2. Fonts

Trang google fonts: <https://fonts.google.com/>

Thuộc tính font-family: Để chỉ định font chữ của văn bản

↴ Ví dụ:

- ↴ font-family: "Times New Roman", Times, serif;
- ↴ font-family: Arial, Helvetica, sans-serif;
- ↴ font-family: "Lucida Console", "Courier New", monospace;

06. Text, Fonts, Icons

6.2. Fonts

font-style: Thuộc tính font-style chủ yếu được sử dụng để chỉ định văn bản in nghiêng.

- ↳ Thuộc tính này có 3 giá trị:
 - ↳ normal - Văn bản được hiển thị bình thường
 - ↳ **italic** - Văn bản được hiển thị in nghiêng
 - ↳ oblique (xiên) - Văn bản "nghiêng" (xiên rất giống với chữ nghiêng, nhưng nghiêng hơn và ít được sử dụng)

06. Text, Fonts, Icons

6.2. Fonts

Font Weight:

- ↴ Thuộc tính font-weight để chỉ định độ dày của chữ
- ↴ Giá trị có thể là: normal, bold, 100, 200, 300,..., 800,900
- ↴ Ví dụ:
 - ↴ font-weight: 700;
 - ↴ font-weight: bold;
 - ↴ font-weight: normal;
 - ↴ font-weight: 400;

06. Text, Fonts, Icons

6.2. Fonts

Font Size:

- ↴ Thuộc tính font-size dùng để đặt kích thước của văn bản
- ↴ Ví dụ:
 - ↴ font-size: 12px;
 - ↴ font-size: 14px;
 - ↴ font-size: 16px;
 - ↴ font-size: 18px;

06. Text, Fonts, Icons

6.3. Icons

Link website: <https://fontawesome.com/search?o=r&m=free>

Link CDN: <https://cdnjs.com/libraries/font-awesome>



Bài tập

Link bài tập:

<https://eng2it.com/course/view.php?id=8>

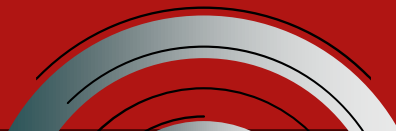


01. List (Danh sách)

list-style-type: chỉ định kiểu đánh dấu mục danh sách.

- **list-style-type: disc;** // Hình tròn đen. Mặc định.
- **list-style-type: circle;** // Hình tròn có viền đen và trống bên trong.
- **list-style-type: square;** // Hình vuông
- **list-style-type: disclosure-closed;** // Hình tam giác trỏ sang phải
- **list-style-type: none;** // Bị ẩn
- **list-style-type: upper-roman;** // Chữ la mã I, II, III
- **list-style-type: lower-alpha;** // Chữ cái a, b, c

Link nghiên cứu thêm: <https://developer.mozilla.org/en-US/docs/Web/CSS/list-style-type>



01. List (Danh sách)

list-style-position: chỉ định vị trí của các điểm đánh dấu mục danh sách.

- **list-style-position: outside;** // Các điểm đánh dấu nằm bên ngoài
- **list-style-position: inside;** // Các điểm đánh dấu nằm bên trong

02. Table (Bảng)

Code các bảng sau, sử dụng HTML, CSS:

Ví dụ 1:

Hàng 1 Cột 1	Hàng 1 Cột 2	Hàng 1 Cột 3
Hàng 1 Cột 2	Hàng 2 Cột 2	Hàng 2 Cột 3
Hàng 3 Cột 1	Hàng 3 Cột 2	Hàng 3 Cột 3

Ví dụ 2:

Hàng 1 Cột 1	Hàng 1 Cột 2	Hàng 1 Cột 3
Hàng 2 Cột 1	Hàng 2 Cột 2	Hàng 2 Cột 3
Hàng 3 Cột 1	Hàng 3 Cột 2	Hàng 3 Cột 3

02. Table (Bảng)

Code các bảng sau, sử dụng HTML, CSS:

Ví dụ 3:

Hàng 1 Cột 1	Hàng 1 Cột 2	Hàng 1 Cột 3
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ut ipsum tempora pariatur voluptatibus reiciendis commodi harum porro minus omnis dignissimos aliquid, in blanditiis nobis nam alias voluptas odio atque aspernatur. Lorem, ipsum dolor sit amet consectetur adipisicing elit. Velit corporis placeat veritatis amet sint mollitia blanditiis culpa vitae incidunt accusantium impedit repudiandae labore magni, ut ipsam odit? Praesentium, ut itaque.	Hàng 2 Cột 2	Hàng 2 Cột 3
Hàng 3 Cột 1	Hàng 3 Cột 2	Hàng 3 Cột 3

02. Table (Bảng)

Code các bảng sau, sử dụng HTML, CSS:

Ví dụ 4:

Hàng 1 Cột 1	Hàng 1 Cột 2	Hàng 1 Cột 3
Hàng 1 Cột 2	Hàng 2 Cột 2	Hàng 2 Cột 3
Hàng 3 Cột 1	Hàng 3 Cột 2	Hàng 3 Cột 3

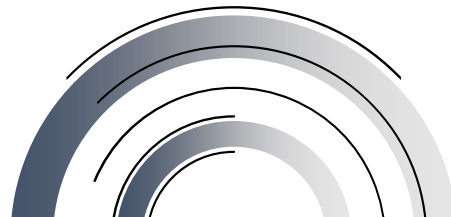
03. Display (Hiển thị)

display: inline; (không đặt được chiều rộng và chiều cao, không thêm được margin và padding cho bên trên và bên dưới)

display: block; (Chiều rộng kéo dài từ trái sang phải, luôn bắt đầu ở một hàng mới. Cho phép đặt chiều rộng và chiều cao, cho phép đặt margin, padding cho bên trên và bên dưới)

display: inline-block; (Chiều rộng mặc định bằng chiều rộng của element, không bắt đầu ở một hàng mới. Cho phép đặt chiều rộng và chiều cao, cho phép đặt margin, padding cho bên trên và bên dưới)

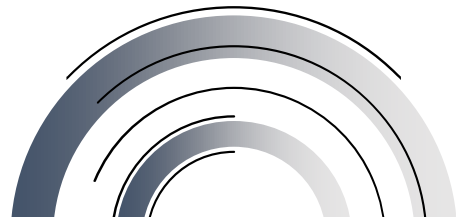
display: none; (Ẩn đi hoàn toàn)



03. Visibility (Hiển thị)

visibility: hidden; // Ẩn nhưng vẫn chiếm một khoảng trống

visibility: visible; // Hiển thị. mặc định.

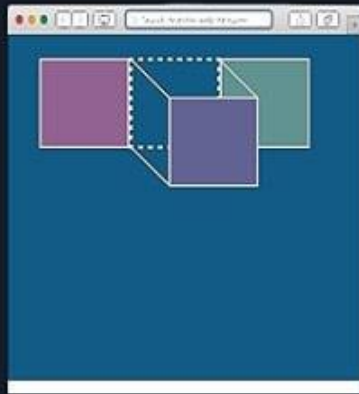


04. Position (Vị trí)

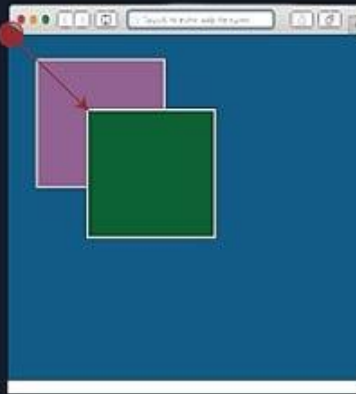
CSS Position Property



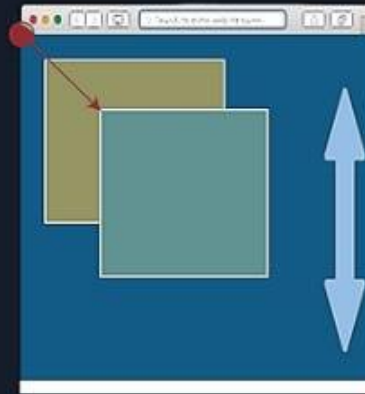
Static



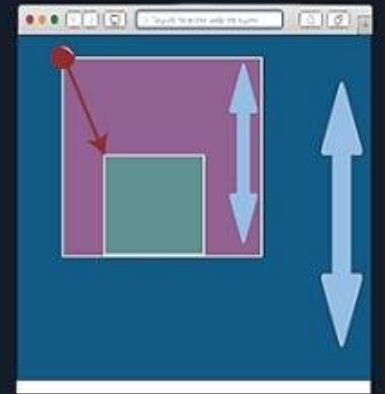
Relative



Absolute



Fixed



Sticky

04. Position (Vị trí)

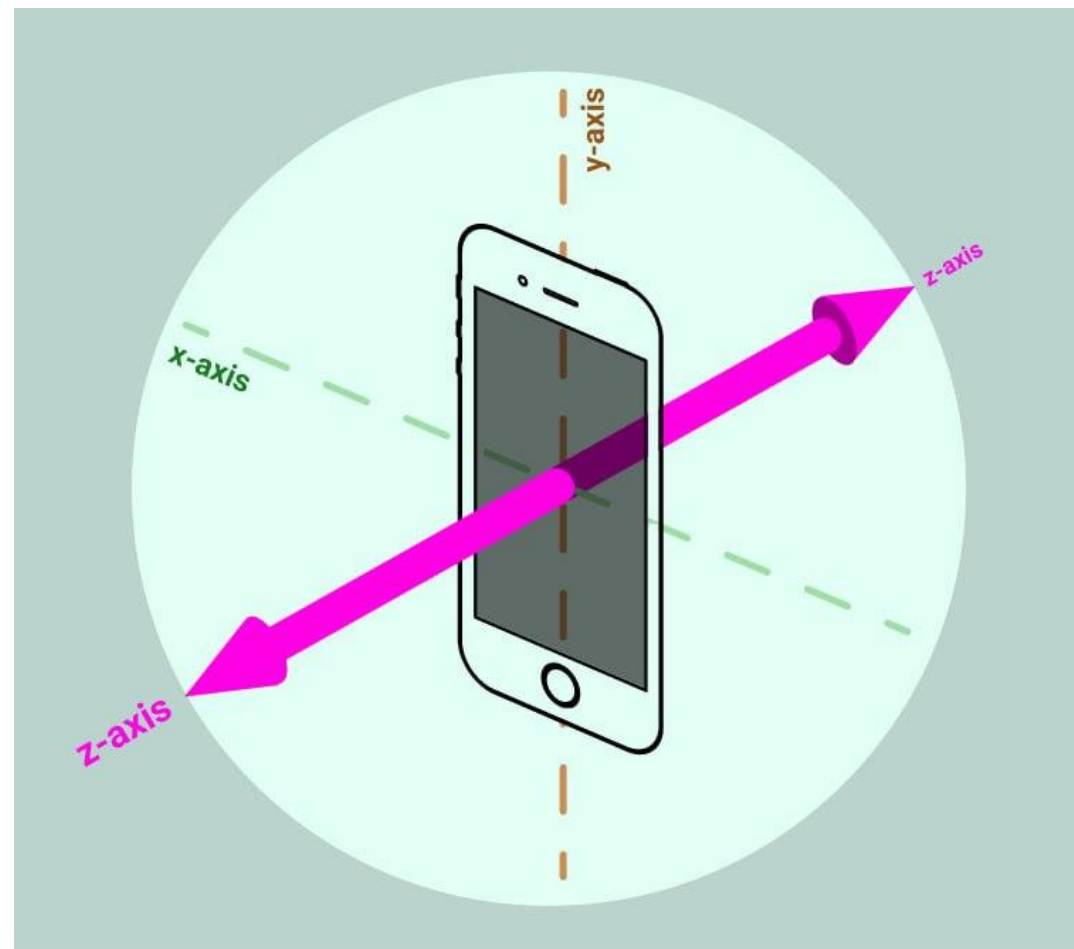
05. Z-index

Thuộc tính z-index thiết lập thứ tự xếp chồng nhau của một thành phần vị trí.

Giá trị mặc định là 0.

z-index càng cao thì element đó càng nằm trên.

Chú ý: z-index chỉ làm việc cùng với thuộc tính position có giá trị sau: absolute, fixed, relative, sticky. Đối với giá trị static thì z-index không áp dụng được.

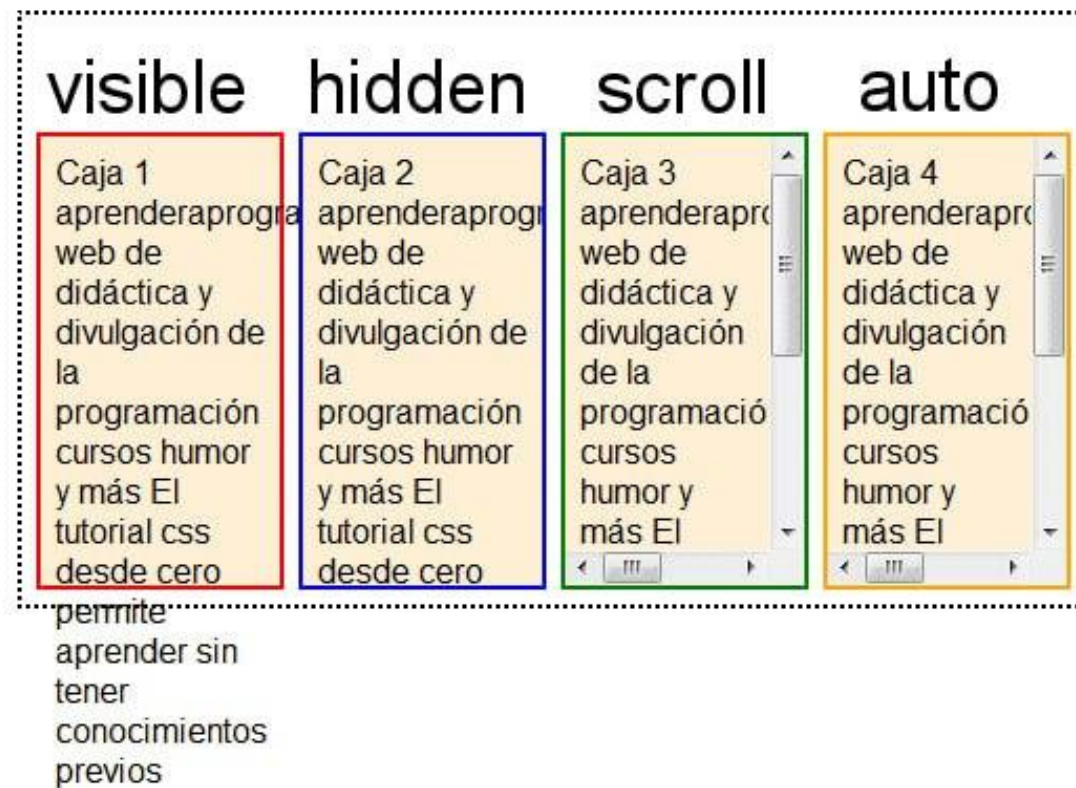


06. Overflow (Tràn ra)

**overflow: visible; // Khi chiều cao của box
không đủ chứa text, thì text vẫn hiển thị
tràn qua box, đây là mặc định.**

**overflow: hidden; // Khi chiều cao của box
không đủ chứa text, thì text bị tràn sẽ
được ẩn đi.**

overflow css



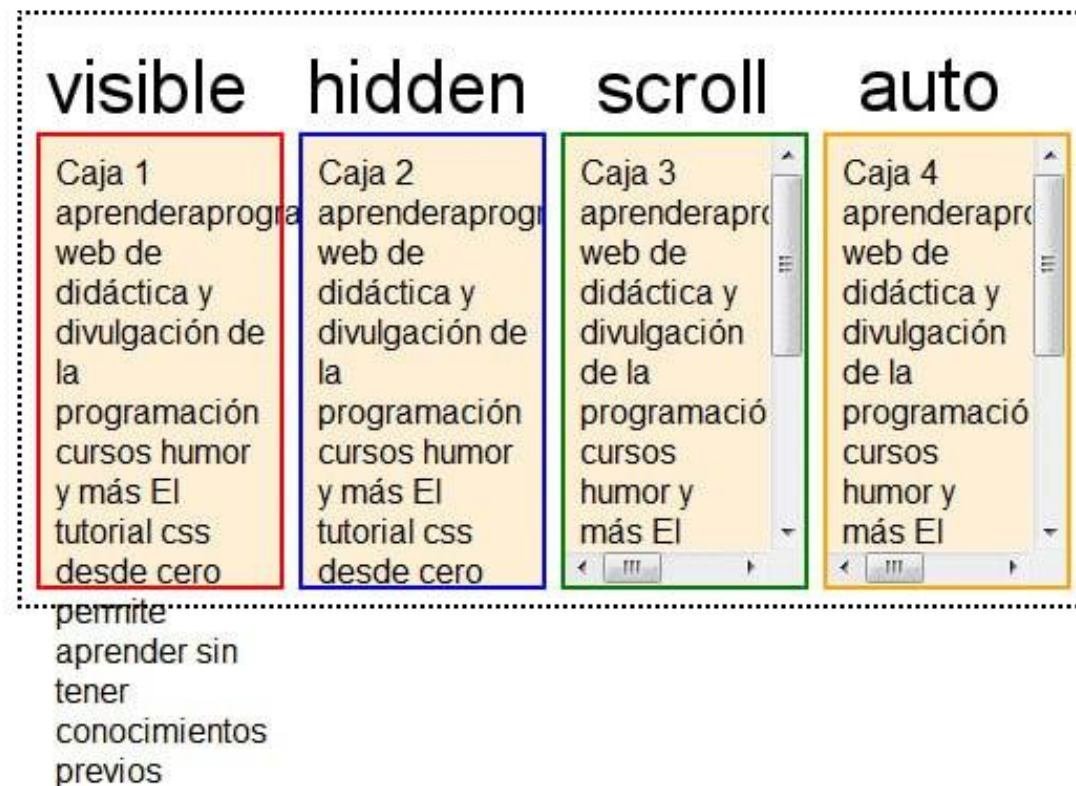
06. Overflow (Tràn ra)

overflow: scroll; // Khi chiều cao của box không đủ chứa text, thì text bị tràn sẽ được ẩn đi và xuất hiện thanh scroll, khi cuộn sẽ hiển thị text. Khi sử dụng thành phần này sẽ xuất hiện cả thanh scroll ngang và dọc.

overflow: auto; Khi chiều cao của box

không đủ chứa text, thì thanh scroll

overflow css

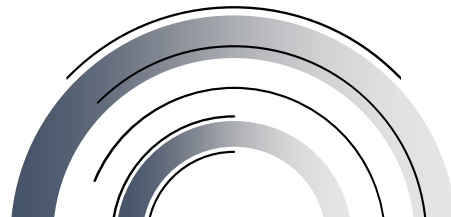


07. Combinator selectors (Bộ chọn tổ hợp)

Chọn phần tử tag02 được đặt cùng cấp và phải kề ngay sau phần tử tag01 (mỗi tag01 chỉ có một tag02 kề ngay sau).

Cú pháp:

- `tag01 + tag02 { /*Viết css trong này*/ }`

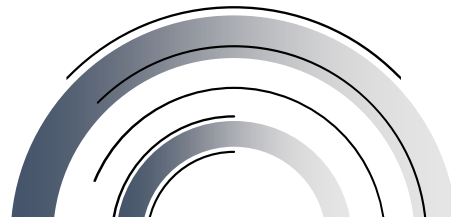


07. Combinator selectors (Bộ chọn tổ hợp)

Chọn tất cả phần tử tag02 cùng cấp với phần tử tag01, và những phần tử tag02 phải nằm sau phần tử tag01.

Cú pháp:

- `tag01 ~ tag02 { /*Viết css trong này*/ }`

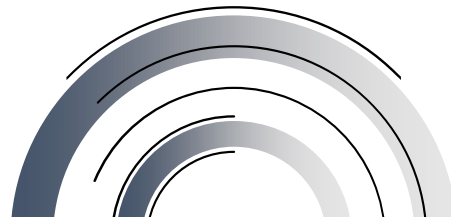


07. Combinator selectors (Bộ chọn tổ hợp)

Chọn tất cả phần tử tag02 là cấp con đầu tiên (cấp 1) của phần tử tag01.

Cú pháp:

- `tag01 > tag02 { /*Viết css trong này*/ }`

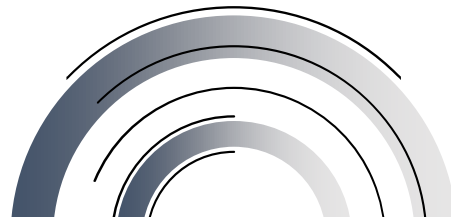


07. Combinator selectors (Bộ chọn tổ hợp)

Chọn tất cả phần tử tag02 bên trong phần tử tag01.

Cú pháp:

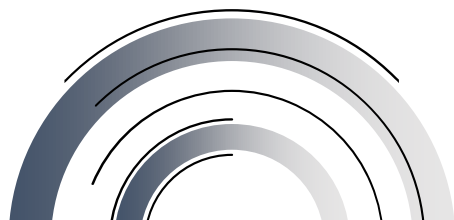
- `tag01 tag02 { /*Viết css trong này*/ }`



08. Pseudo-class selectors (Bộ chọn lớp giả)

Cú pháp:

```
selector:pseudo-class {  
  property: value;  
}
```



08. Pseudo-class selectors (Bộ chọn lớp giả)

:link: khi liên kết chưa được truy cập lần nào. (Chỉ dùng cho thẻ a)

:visited: khi liên kết đã được truy cập sau lần đầu tiên. (Chỉ dùng cho thẻ a)

:hover: khi di chuyển chuột lên element.

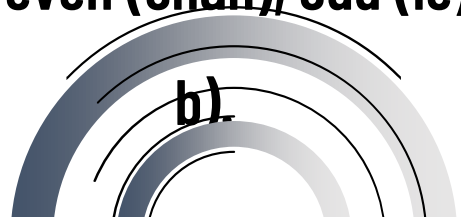
:active: khi phần tử được click vào, áp dụng cho tất cả các thẻ.

:first-child: thiết lập thuộc tính cho element đầu tiên.

:last-child: thiết lập thuộc tính cho element cuối cùng.

:nth-child(n): n có thể là các giá trị even (chẵn), odd (lẻ), một số, một biểu thức ($an +$

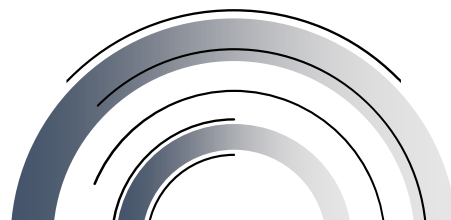
b).



09. Pseudo-elements selectors (Bộ chọn phần tử giả)

Cú pháp:

```
selector::pseudo-element {  
  property: value;  
}
```



09. Pseudo-elements selectors (Bộ chọn phần tử giả)

::before: Chèn nội dung nào đó trước nội dung của mỗi element. (Phải thêm content: "";))

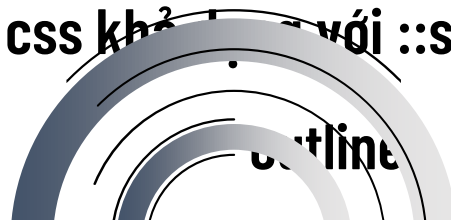
::after: Chèn nội dung nào đó sau nội dung của mỗi element. (Phải thêm content: "";))

::first-letter: Chọn chữ cái đầu tiên của mỗi element.

::first-line: Chọn dòng đầu tiên của mỗi element.

::marker: Chọn các điểm đánh dấu của các mục danh sách (thẻ và)

::selection: Được dùng để style cho một vùng văn bản được người dùng chọn (hay còn gọi là "bôi đen"). Chỉ có một số thuộc tính css khác nhau với ::selection là color, background, cursor, và



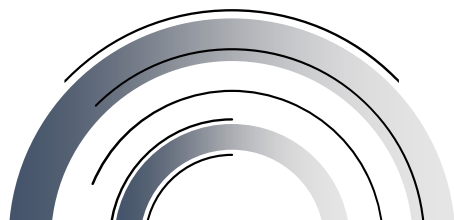
10. Opacity (Độ mờ)

Thuộc tính opacity chỉ định độ mờ/độ trong suốt của một element.

Thuộc tính opacity có thể nhận giá trị từ 0 đến 1. Giá trị càng thấp thì element càng mờ.

Cú pháp:

- `opacity: 0.6; // Element sẽ mờ đi 40%`



- Bài tập

<https://eng2it.com/course/view.php?id=8>



Nội dung

01

Float và Clear

03

Units (Đơn vị)

05

Math Functions (Hàm toán học)

07

Variables (Biến)

09

Khái niệm Responsive

02

Attribute selectors (Bộ chọn thuộc tính)

04

Specificity và !important

06

CSS function attr()

08

Box Sizing (Kích thước hộp)

10

Media Queries

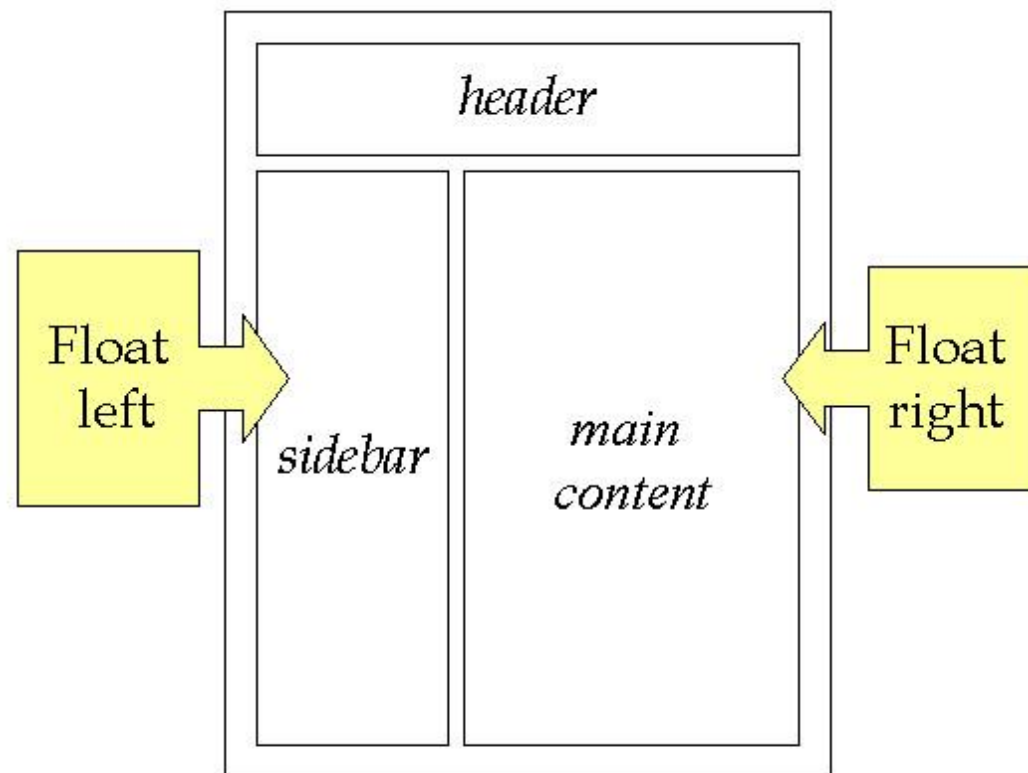
01. Float và Clear

1.1. Float

Thuộc tính float là một thuộc tính được sử dụng để chuyển một phần tử sang góc trái hoặc góc bên phải của không gian bao quanh nó.

Float có các giá trị sau:

- float: left; // Phần tử nằm phía bên trái.
- float: right; // Phần tử nằm phía bên phải.
- float: none; // Phần tử nằm tại chính vị trí của nó (Mặc định).



01. Float và Clear

1.2 Clear

Thuộc tính Clear là một thuộc tính ngược lại hoàn toàn với Float.

Ngăn chặn phần tử B chiếm vùng không gian của phần tử A (với A là phần tử sử dụng float).

Thuộc tính clear có các giá trị sau:

- clear: left;
 - Ngăn chặn phần tử B chiếm chỗ phần tử A, khi phần tử A có sử dụng float: left;
 - Thuộc tính clear: left; không có tác dụng khi phần tử A sử dụng float: right;
- clear: right;
 - Ngăn chặn phần tử B chiếm chỗ phần tử A, khi phần tử A có sử dụng float: right;
 - Thuộc tính clear: right; không có tác dụng khi phần tử A sử dụng float: left;



01. Float và Clear

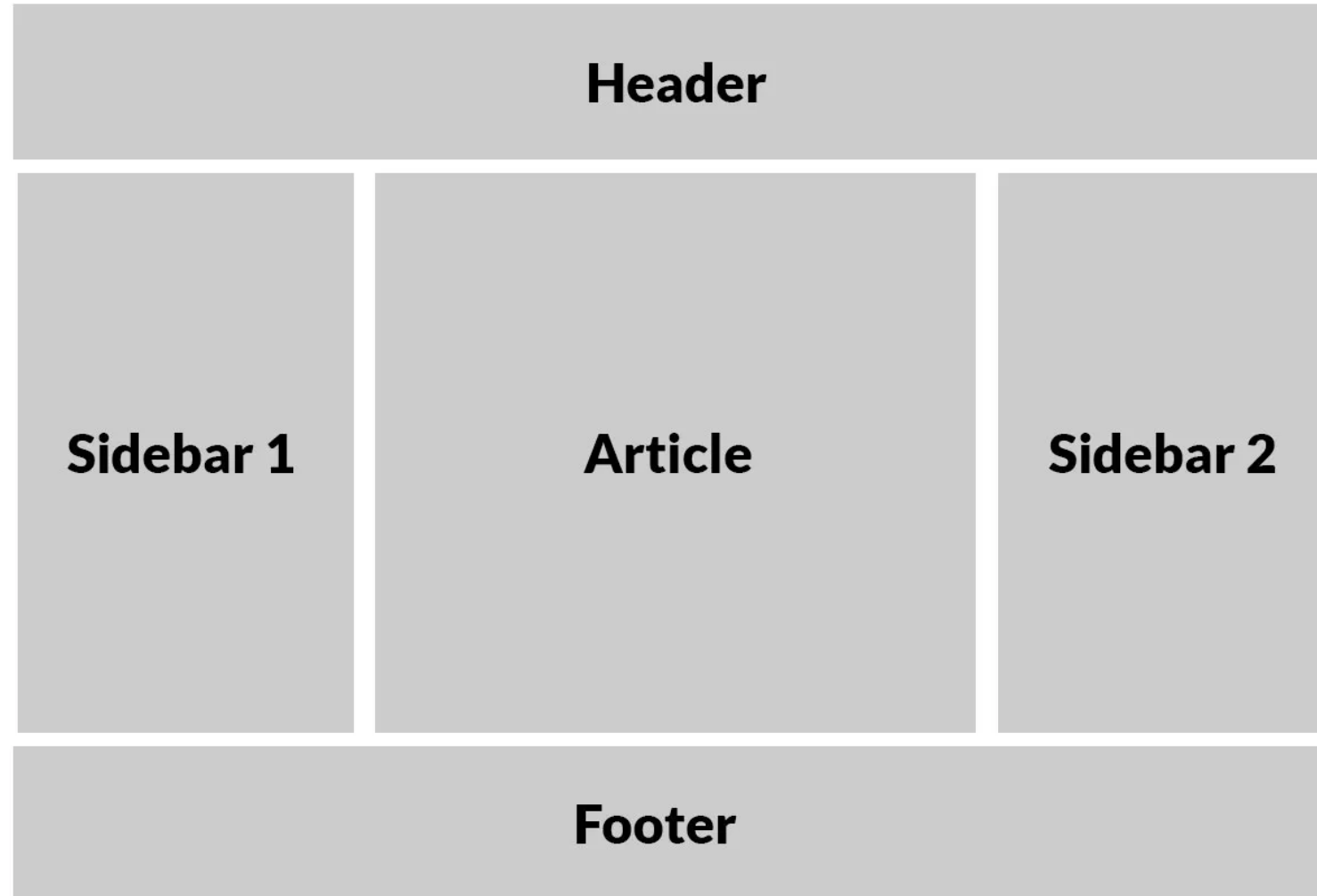
1.2 Clear

Thuộc tính clear có các giá trị sau

- clear: both;
 - Khi cả 2 phần tử A và phần tử B sử dụng float, chúng ta không thể sử dụng clear: left; hay clear: right; để ngăn chặn thành phần C chiếm vùng không gian còn trống, trong trường hợp này ta sử dụng thuộc tính clear: both; để ngăn chặn sự chiếm vùng của thành phần C.
 - Thuộc tính clear: both; có thể ngăn chặn sự chiếm vùng không gian cả khi chỉ có một phần tử sử dụng float (left hoặc right).
- clear: none;
 - Thuộc tính clear: none; là dạng phục hồi khi Thành phần sử dụng thuộc tính clear.

01. Float và Clear

1.3. Ví dụ



02 Attribute selectors (Bộ chọn thuộc tính)

Bộ chọn attribute được sử dụng để chọn các phần tử có thuộc tính được chỉ định.

Một số kiểu chọn:

[attribute]

- Chọn tất cả các phần tử có thuộc tính [attribute].

[attribute="value"]

- Chọn tất cả các phần tử có thuộc tính [attribute] phải chứa giá trị value.
- Chỉ được có giá trị duy nhất là value, nếu có thêm giá trị khác thì sẽ không được chọn.
- Giá trị có thể bao gồm nhiều từ.



02 Attribute selectors (Bộ chọn thuộc tính)

Bộ chọn attribute được sử dụng để chọn các phần tử có thuộc tính được chỉ định.

Một số kiểu chọn:

`[attribute~="value"]`

- Chọn tất cả các phần tử có thuộc tính `[attribute]` phải chứa giá trị `value`.
- Nhưng `value` đó phải đứng độc lập, không được viết liền với từ khác.
- `value` phải là 1 từ khóa duy nhất.

`[attribute|="value"]`

- Chọn các phần tử có thuộc tính `[attribute]`, giá trị có thể chính xác là giá trị được chỉ định hoặc giá trị được chỉ định theo sau dấu gạch nối (-).
- Lưu ý: Giá trị phải là một từ nguyên vẹn, đơn lẻ như `class="top"` hoặc theo sau là dấu gạch ngang (-), như `class="top-text"`.

02 Attribute selectors (Bộ chọn thuộc tính)

Bộ chọn attribute được sử dụng để chọn các phần tử có thuộc tính được chỉ định.

Một số kiểu chọn:

`[attribute^="value"]`

- Chọn tất cả các phần tử có thuộc tính [attribute] bắt đầu bằng giá trị value
- Khác với `[attribute="value"]` ở chỗ nó không có ngoại lệ và chọn tất cả các phần tử bắt đầu bằng value, được phép viết liền với các từ khác.

`[attribute$="value"]`

- Chọn tất cả các phần tử có thuộc tính [attribute] kết thúc bằng giá trị value.
- Lưu ý: Giá trị có thể được viết liền với từ khác.

02 Attribute selectors (Bộ chọn thuộc tính)

Bộ chọn attribute được sử dụng để chọn các phần tử có thuộc tính được chỉ định.

Một số kiểu chọn:

`[attribute*='value']`

- Chọn tất cả các phần tử có thuộc tính [attribute] chứa giá trị value.
- Lưu ý: Giá trị không nhất thiết phải là một từ nguyên vẹn!

03. Units (Đơn vị)

Độ dài tuyệt đối

Là loại đơn vị có giá trị cố định và thể hiện chính xác chiều dài kích thước nó hiển thị, không phụ thuộc cũng như không thay đổi bởi bất kỳ tác động bên ngoài nào.

Một số đơn vị:

- cm
- mm
- in: inches (1in = 96px = 2.54cm)
- px: pixels (1px = 1/96 in) tương ứng với một điểm ảnh trên máy tính.
- pt: points (1pt = 1/72 in)
- pc: picas (1pc = 12 pt)

03. Units (Đơn vị)

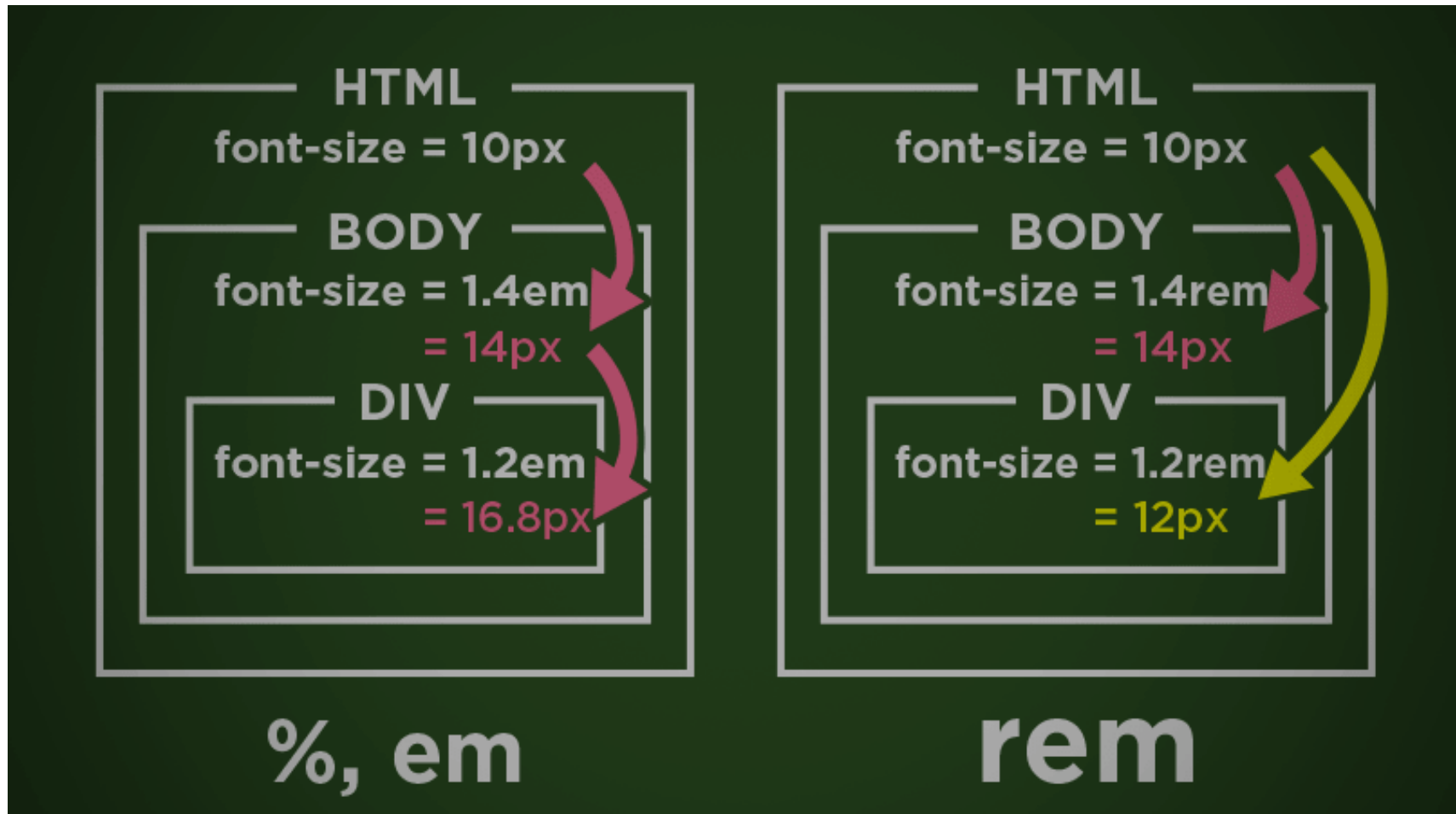
Độ dài tương đối

Là đơn vị đo lường được sử dụng trong CSS ở mức tương đối, thường phụ thuộc vào 1 thành phần nào đó thì mới xác định được giá trị của nó.

Một số đơn vị:

- %: giá trị tương đối so với phần tử cha.
- rem giá trị tương đối với font-size của phần tử gốc. Phần tử gốc ở đây là thẻ html.
- em 1em tương đương với kích cỡ của font-size của phần tử cha có định nghĩa font-size.
- vw (viewport width): 1vw tương đương với 1% của chiều rộng của sổ trình duyệt.
- vh (viewport height): 1vh tương đương với 1% của chiều cao của sổ trình duyệt.
- ex: 1ex tương đương với chiều cao (height) của một chữ x (in thường) của font hiện tại.
- ch: 1ch tương đương với chiều rộng (width) của một số 0 theo size hiện tại.

03. Units (Đơn vị)

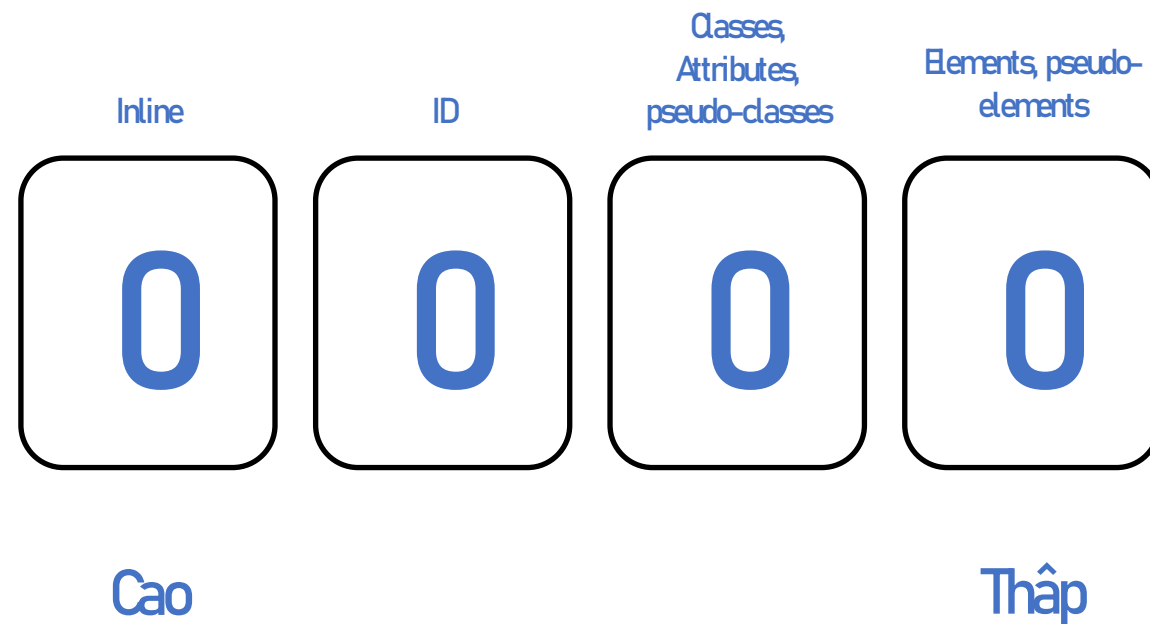


04. Specificity (Tính đặc hiệu) và !important (Quan trọng)

4.1. Specificity (Tính đặc hiệu)

Tính đặc hiệu (hay độ ưu tiên) là cách mà trình duyệt quyết định sẽ áp dụng thuộc tính css nào với một phần tử khi có nhiều quy tắc css cùng trỏ đến phần tử đó.

Sự phân cấp tính đặc hiệu. Mỗi selector đều có vị trí của nó trong hệ thống phân cấp. Độ đặc hiệu của selector có 4 mức (như hình):



04. Specificity (Tính đặc hiệu) và !important (Quan trọng)

4.1. Specificity (Tính đặc hiệu)

Làm sao để tính toán tính đặc hiệu? Ta biểu diễn tương đối tính đặc hiệu của một selector như dưới đây, sau đó xem giá trị nào càng cao thì càng được ưu tiên:

- 1-0-0-0: Inline styles
- 0-X-0-0: Số lượng ID selector
- 0-0-Y-0: Số lượng Classes, attributes và pseudo-classes
- 0-0-0-Z: Số lượng Elements và pseudo-elements
- Viết CSS theo kiểu Internal và External không có độ ưu tiên.
- Universal selector (*) và combinators selector (+, >, ~) không làm tăng tính đặc hiệu.

04. Specificity (Tính đặc hiệu) và !important (Quan trọng)

4.1. Specificity (Tính đặc hiệu)

Ví dụ: Tính toán tính đặc hiệu của các bộ chọn sau:

- `h2 { color: red; }`
- `h2.title { color: blue; }`
- `div h2.title { color: orange; }`
- `.head h2.title { color: green; }`
- `div.head h2.title {color: gray; }`
- `div.head h2.title:last-child {color: yellow; }`
- `#main div.head h2.title {color: pink; }`
- `#main div#head h2.title {color: purple; }`
- `#main div#head h2.title::first-letter {color: red; }`
- `#main div#head h2::first-letter {color: yellow; }`

Code HTML mẫu:

```
<main id="main">
  <div class="head"
id="head">
    <h2 class="title">
      Tiêu đề 1
    </h2>
    <h2 class="title">
      Tiêu đề 2
    </h2>
  </div>
</main>
```

04. Specificity (Tính đặc hiệu) và !important (Quan trọng)

4.2 !important (Quan trọng)

Important được sử dụng để thay đổi thứ tự ưu tiên của CSS, khi một thuộc tính CSS nào đó được gán lệnh Important thì đồng nghĩa với việc nó sẽ có mức ưu tiên cao nhất. Nếu có nhiều thuộc tính cùng có !important thì lại quay về bài toán tính độ ưu tiên.

Ví dụ:

- `h2 { color: red !important; }`
- `h2.title { color: blue; }`
- `div h2.title { color: orange; }`
- `.head h2.title { color: green; }`
- `div.head h2.title { color: gray; }`
- `#main div.head h2.title { color: pink; }`

05. Math Functions (Hàm toán học)

Các hàm toán học CSS cho phép các biểu thức toán học được sử dụng làm giá trị thuộc tính.

Các hàm toán học:

Hàm `calc()`

- Thực hiện một phép tính, kết quả sẽ lấy làm giá trị thuộc tính. Các toán tử sau có thể được sử dụng: `+` `-` `*` `/`
- Ví dụ: `width: calc(100% - 100px);`

Hàm `max()`

- Sử dụng giá trị lớn nhất, từ danh sách giá trị được phân tách bằng dấu phẩy, làm giá trị thuộc tính.
- cú pháp: `max(value1, value2, ...)`
- Ví dụ: `width: max(50%, 300px);`

05. Math Functions (Hàm toán học)

Các hàm toán học CSS cho phép các biểu thức toán học được sử dụng làm giá trị thuộc tính.

Các hàm toán học:

Hàm `min()`

- Sử dụng giá trị nhỏ nhất, từ danh sách giá trị được phân tách bằng dấu phẩy, làm giá trị thuộc tính.
- cú pháp: `min(value1, value2, ...)`
- Ví dụ: `width: min(50%, 300px);`

06. CSS function attr()

`attr()` là một hàm CSS trả về giá trị của một thuộc tính

Ví dụ:

```
<style>
  [data-text]::before {
    content: attr(data-text);
  }
</style>
```

```
<span data-text="Xin chào ">Đăng Phương Nam</span>
```

07. Variables (Biến)

Mục đích của khai báo biến là để có thể sử dụng được ở nhiều nơi.

Kiểu global (toàn cục): có thể sử dụng được nhiều nơi trong file css.

Kiểu local (cục bộ): chỉ sử dụng được ở trong phạm vi của nó, những đoạn css khác không sử dụng được.

07. Variables (Biến)

Cách tạo biến global:

```
:root {  
    --ten-bien: giá trị;  
}
```

Cách sử dụng biến:

```
var(--ten-bien);
```

Ví dụ:

```
:root {  
    --gray: #333333;  
    --white: #ffffff;  
}  
  
body {  
    color: var(--gray);  
    background-color: var(--white);  
}
```


07. Variables (Biến)

Cách tạo biến local:

```
selector {  
  --ten-bien: giá trị;  
}
```

Cách sử dụng biến:

```
selector {  
  --ten-bien: giá trị;  
  var(--ten-bien);  
}
```

Ví dụ:

```
h1 {  
  --local-color: blue;  
  color: var(--local-color);  
}  
  
p {  
  color: var(--local-color);  
}
```

08. Box Sizing (Kích thước hộp)

Mặc định khi ta sử dụng thuộc tính width, height thì chỉ là áp dụng cho phần content của 1 element.

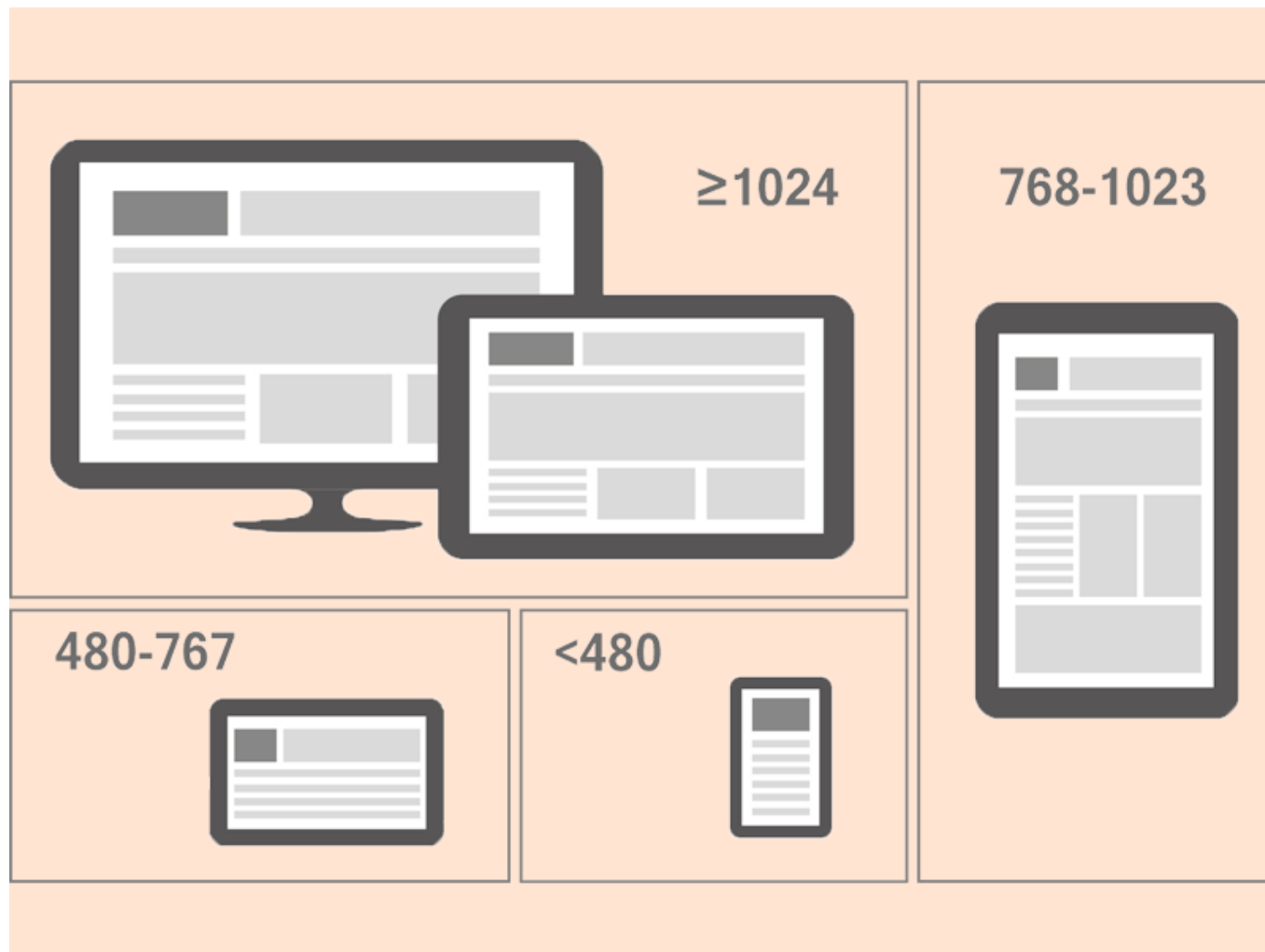
2 loại box-sizing:

- content-box:
 - Mặc định. Width/height chỉ bao gồm “nội dung” của phần tử.
 - $\text{width, height} = \text{content}$
- border-box:
 - Width/height bao gồm content, padding, border của phần tử.
 - $\text{width, height} = \text{content} + \text{padding} + \text{border}$.

09. Khái niệm Responsive

Responsive là để chỉ một website có thể hiển thị tương thích trên mọi thiết bị, như máy tính, máy tính bảng, điện thoại.

Sử dụng media query để responsive giao diện.



09. Khái niệm Responsive

Khai báo meta viewport:

- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- Trong đó
 - viewport: Giúp trình duyệt hiểu thẻ meta này dùng để thiết lập khung nhìn.
 - width=device-width: Đặt chiều rộng bằng chiều rộng thiết bị (device-width).
 - initial-scale=1.0: Mức thu phóng của website. initial-scale thường được đặt bằng 1 (có thể tăng giá trị lên nhưng không được khuyến nghị).

10. Media Queries

@media CSS là một tính năng mới của CSS3.

Tính năng này cho phép ta tùy chỉnh CSS cho nhiều thiết bị khác nhau từ máy tính cho đến ipad, điện thoại và các thiết bị in ấn.

Cú pháp:

```
@media mediaType and (mediaFeature) {  
    // Code  
}
```

Trong đó:

- mediaType gồm các thuộc tính hay sử dụng sau:
 - all: Dùng cho mọi thiết bị
 - print: Dùng cho máy in
 - screen: Dùng cho máy tính và các thiết bị di động
- mediaFeature gồm các thuộc tính hay sử dụng sau:
 - max-width: Chiều rộng tối đa của viewport
 - min-width: Chiều rộng tối thiểu của viewport

10. Media Queries

PC first (Responsive):

- PC first là khái niệm để chỉ việc lập trình giao diện từ màn hình to đến màn hình nhỏ.
- Để làm việc với mô hình này chúng ta sử dụng max-width trong media query.
- Dưới đây là các media query điển hình:



Responsive Design

```
<style>
  /*Ipad ngang(1024 x 768)*/
  @media screen and (max-width: 1024px) {
    /* Code */
  }
  /*Ipad dọc(768 x 1024)*/
  @media screen and (max-width: 768px) {
    /* Code */
  }
  /* Smart phone (480 x 640)*/
  @media screen and (max-width: 480px) {
    /* Code */
  }
  /* Smart phone nhỏ */
  @media screen and (max-width: 320px) {
    /* Code */
  }
</style>
```

10. Media Queries

Mobile first:

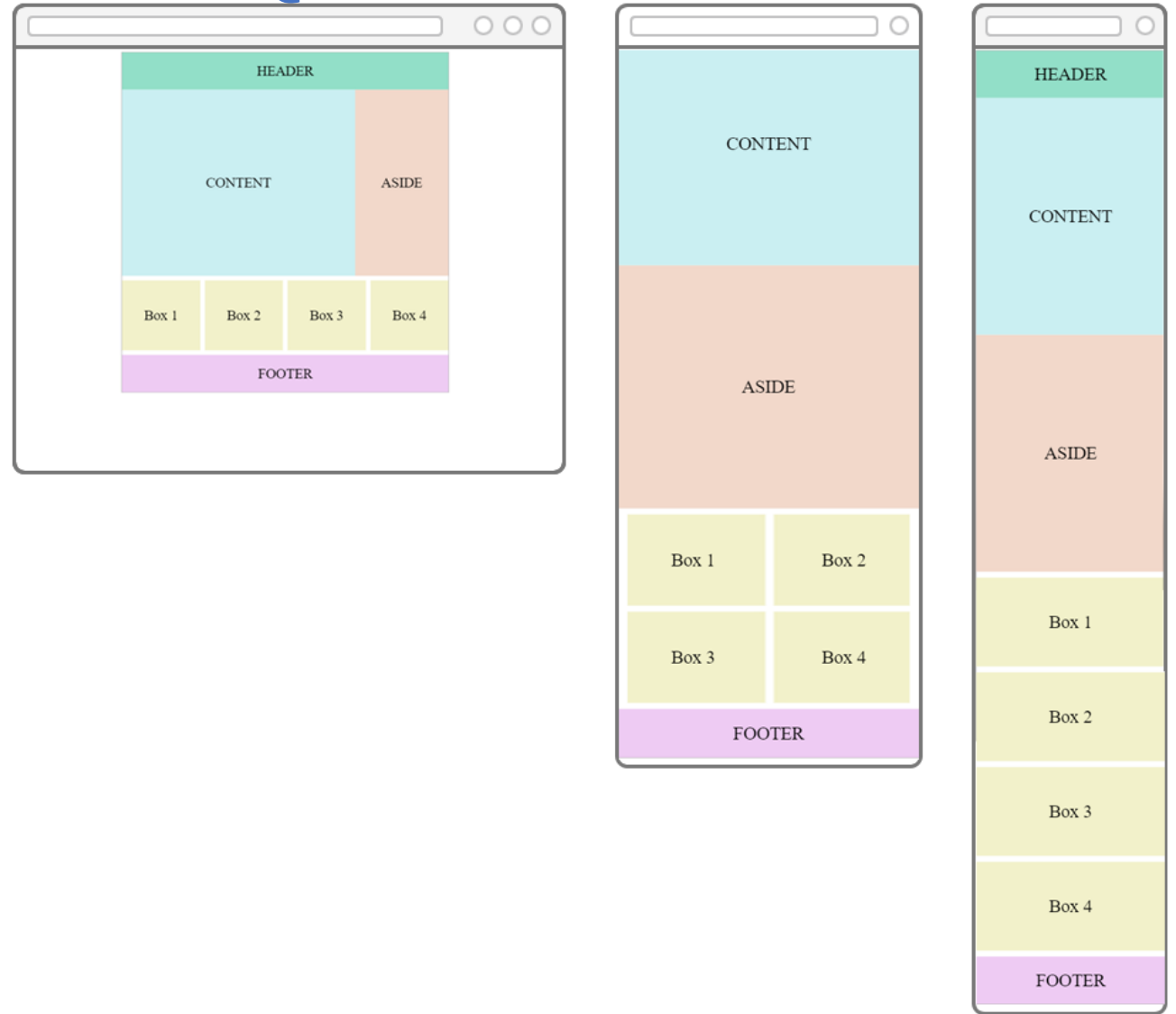
- Mobile first là khái niệm để chỉ việc lập trình giao diện từ màn hình nhỏ đến màn hình to.
- Để làm việc với mô hình này chúng ta sử dụng min-width trong media query.
- Dưới đây là các media query điển hình:



Mobile-First Design

```
<style>
  /* Smart phone nhỏ */
  @media screen and (min-width: 320px) {
    /* Code */
  }
  /* Smart phone nhỏ(480 x 640) */
  @media screen and (min-width: 480px) {
    /* Code */
  }
  /* Ipad dọc(768 x 1024) */
  @media screen and (min-width: 768px) {
    /* Code */
  }
  /* Ipad ngang(1024 x 768) */
  @media screen and (min-width: 1024px) {
    /* Code */
  }
</style>
```

10. Media Queries



Nội dung tổng kết CSS Cơ bản

- **3.1. Giới thiệu về CSS**
- **3.2. Bộ chọn và cú pháp CSS**
- **3.3. Mô hình hộp và cách bố trí**
- **3.4. Định vị CSS và thiết kế đáp ứng**



Nội dung tổng kết CSS Nâng cao

- 4.1. Kỹ thuật bố cục CSS nâng cao
- 4.2. Hoạt ảnh và chuyển tiếp CSS
- 4.3. Bộ tiền xử lý (SASS hoặc LESS)
- 4.4. Khung thiết kế đáp ứng (Bootstrap, Foundation)

